

Experiment No 03

Aim: Create a Cryptocurrency using Python and perform mining in the Blockchain created

Theory:

1. Blockchain Overview

Blockchain is a distributed and decentralized ledger that stores information in a series of linked blocks.

Each block contains:

- Transaction data
- Timestamp
- Previous block's hash
- Its own unique hash (digital fingerprint)

Once data is recorded in a blockchain, it becomes immutable because altering one block would require recalculating all subsequent blocks.

2. Mining

Mining is the process of:

1. Collecting pending transactions into a block.
2. Performing a computational puzzle (Proof-of-Work) to find a valid hash.
3. Adding the new block to the blockchain and broadcasting it to all connected peers.

Miners are rewarded with cryptocurrency for successfully mining a block.

3. Multi-Node Blockchain Network

In this lab, we simulate three independent blockchain nodes (5001, 5002, 5003).

Each node:

- Runs on a separate port.
- Maintains its own copy of the blockchain.
- Can connect with peers to share and validate blocks.

4. Consensus Mechanism

We use the Longest Chain Rule:

- If multiple versions of the chain exist, the longest valid chain is chosen.
- This ensures all nodes agree on a single transaction history.

5. Transactions & Mining Reward

Each transaction has:

- Sender
- Receiver
- Amount

When mining a block:

1. Pending transactions are added to the block.
2. A reward transaction is added automatically to pay the miner.

6. Chain Replacement

When /replace_chain is called:

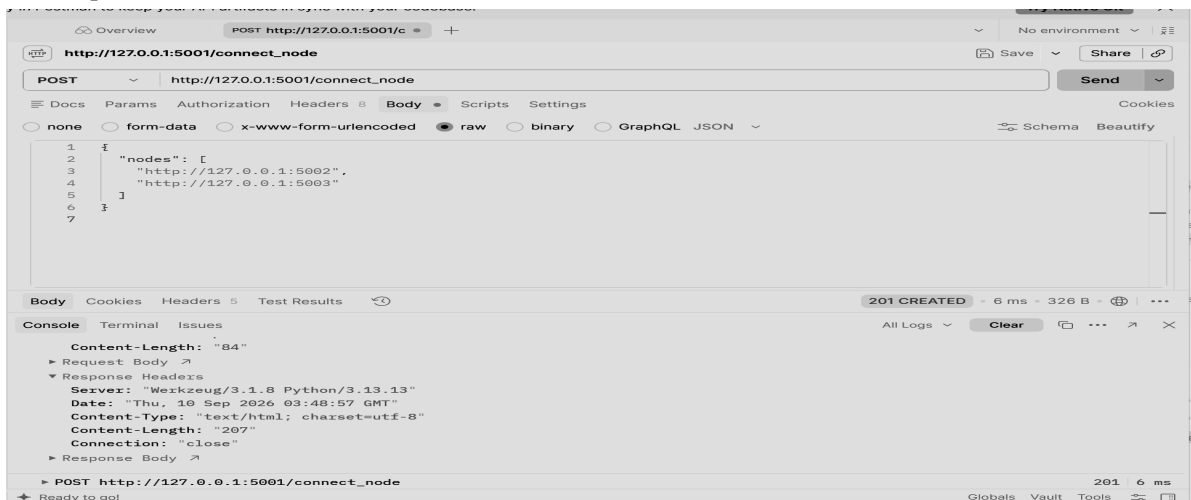
1. Node requests chains from peers.
2. If it finds a longer and valid chain, it replaces its own.
3. This keeps the blockchain consistent across all nodes.

Tools & Libraries Used

- Python 3.x
- Flask – Web framework for API endpoints (pip install Flask)
- Requests – For HTTP communication between nodes (pip install requests==2.18.4)
- Postman – For testing API requests

- Python Standard Libraries: datetime, jsonify, hashlib, uuid4, urlparse

Post request from 5001 to other nodes



* Serving Flask app 'hadcoin_node_5001'

* Debug mode: off

WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.

* Running on all addresses (0.0.0.0)

* Running on http://127.0.0.1:5001

* Running on http://172.20.10.2:5001

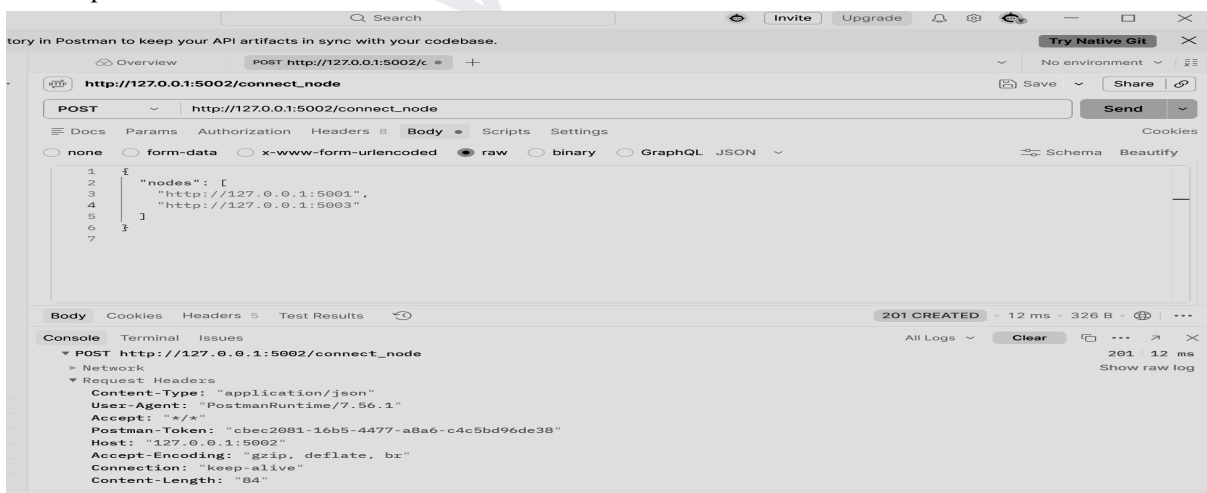
Press CTRL+C to quit

127.0.0.1 - - [10/Sep/2026 09:18:23] "POST /connect_node HTTP/1.1" 401 -

127.0.0.1 - - [10/Sep/2026 09:18:57] "POST /connect_node HTTP/1.1" 401 -

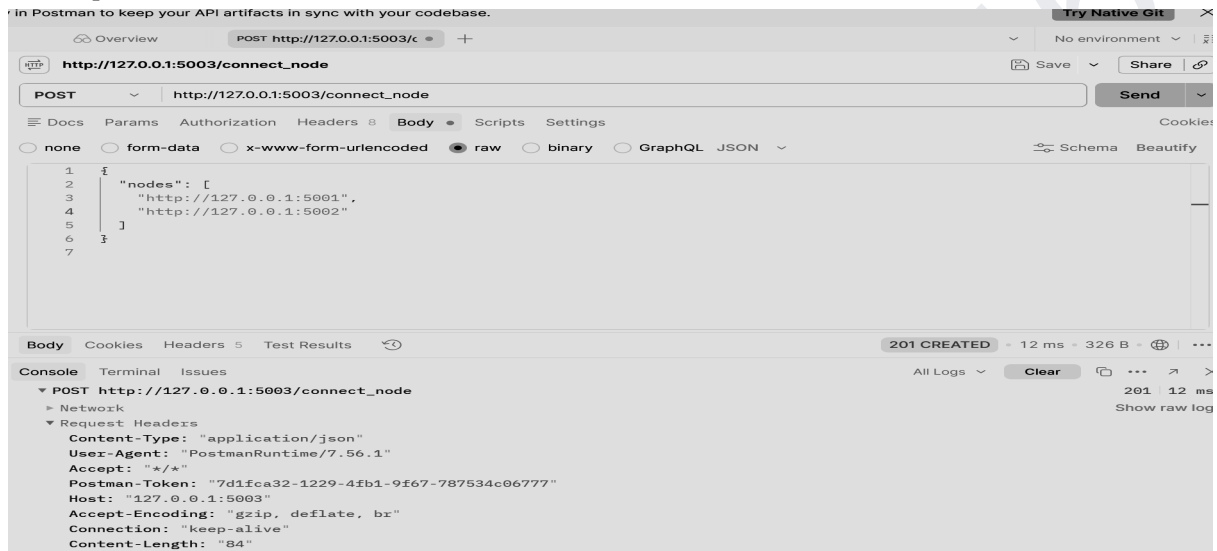
127.0.0.1 - - [10/Sep/2026 09:19:46] "POST /connect_node HTTP/1.1" 201 -

Post request from 5002



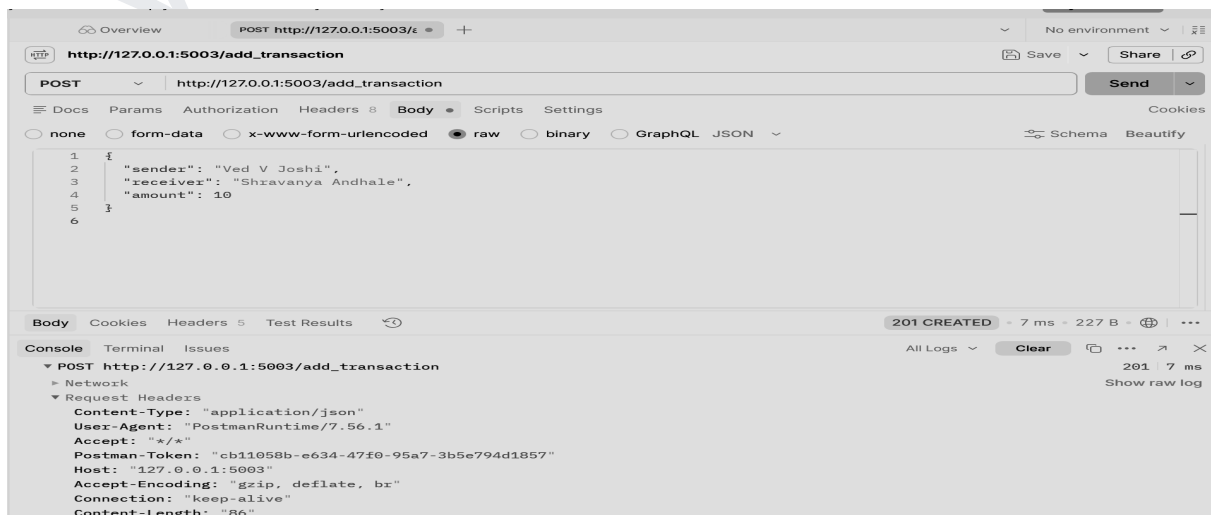
```
* Serving Flask app 'hadcoin_node_5002'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5002
* Running on http://172.20.10.2:5002
Press CTRL+C to quit
127.0.0.1 - - [10/Sep/2026 09:22:01] "POST /connect_node HTTP/1.1" 201 -
```

Post request from 5003



```
* Serving Flask app 'hadcoin_node_5003'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5003
* Running on http://172.20.10.2:5003
Press CTRL+C to quit
127.0.0.1 - - [10/Sep/2026 09:23:01] "POST /connect_node HTTP/1.1" 201 -
```

Add_Transactions
Mine_Transaction



http://127.0.0.1:5003/mine_block

GET http://127.0.0.1:5003/mine_block

Body

200 OK - 8 ms - 597 B

```
{
  "index": 2,
  "message": "Congratulations, you just mined a block!",
  "previous_hash": "5ca9064bf967aa0e45d31bd93bc958eca8b1b23cdc3199c9f35d96f8fd3aca6",
  "proof": 533,
  "timestamp": "2026-09-10 09:27:00.134228",
  "transactions": [
    {
      "amount": 10,
      "receiver": "Shravanya Andhale",
      "sender": "Ved V Joshi"
    },
    {
      "amount": 5,
      "receiver": "Nikhil Kadam",
      "sender": "Shravanya Andhale"
    },
    {
      "amount": 1,
      "receiver": "Abcde",
      "sender": "174eb42e7bf04a95b26ab32f07c27731"
    }
  ]
}
```

Get_Chain

http://127.0.0.1:5003/get_chain

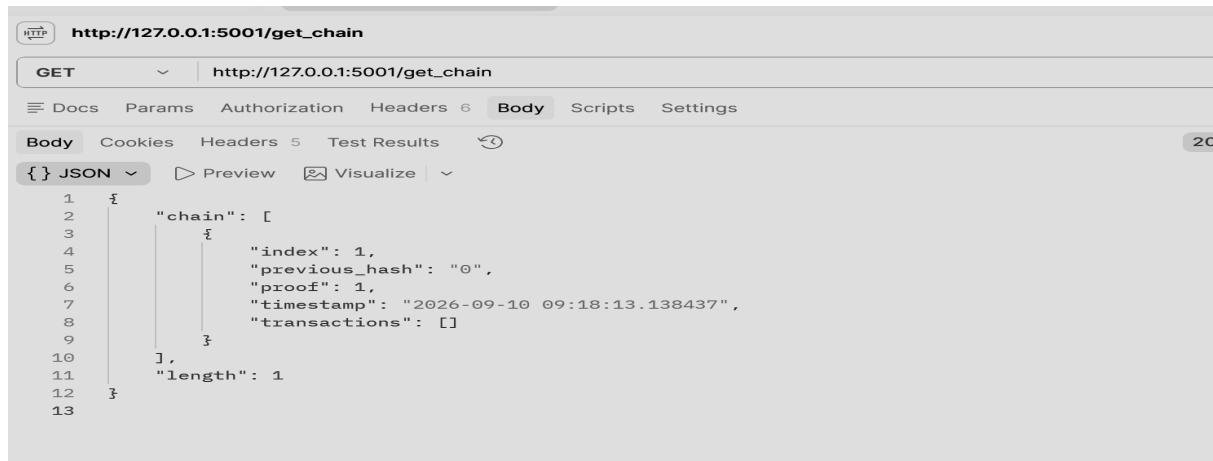
GET http://127.0.0.1:5003/get_chain

Body

200 OK - 5 ms - 668 B

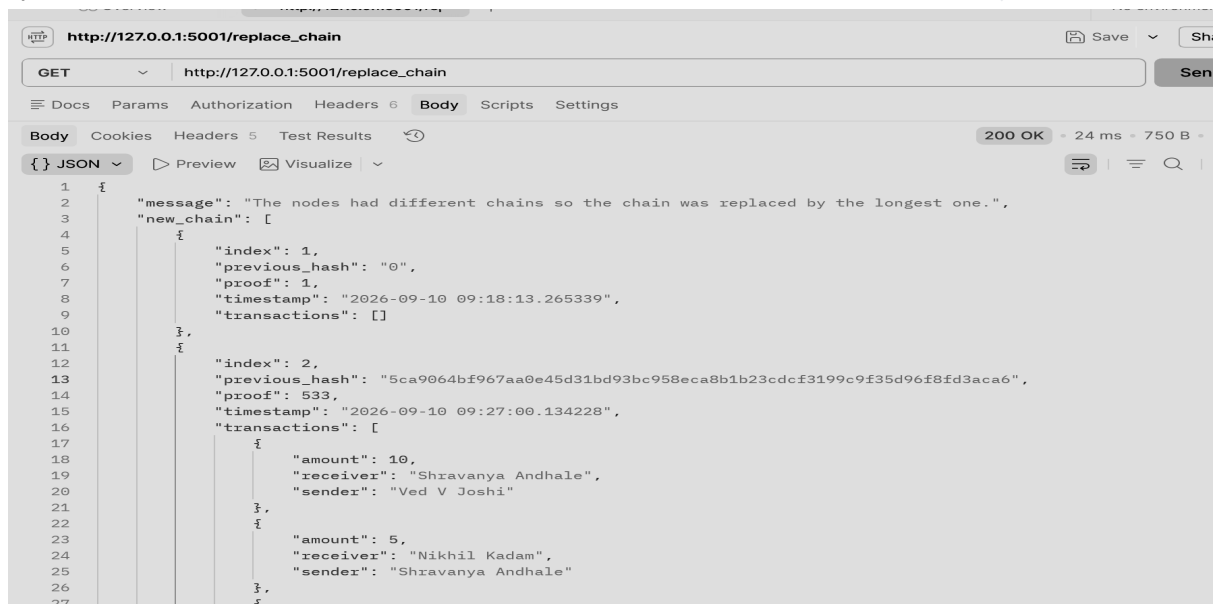
```
{
  "chain": [
    {
      "index": 1,
      "previous_hash": "0",
      "proof": 1,
      "timestamp": "2026-09-10 09:18:13.265339",
      "transactions": []
    },
    {
      "index": 2,
      "previous_hash": "5ca9064bf967aa0e45d31bd93bc958eca8b1b23cdc3199c9f35d96f8fd3aca6",
      "proof": 533,
      "timestamp": "2026-09-10 09:27:00.134228",
      "transactions": [
        {
          "amount": 10,
          "receiver": "Shravanya Andhale",
          "sender": "Ved V Joshi"
        },
        {
          "amount": 5,
          "receiver": "Nikhil Kadam",
          "sender": "Shravanya Andhale"
        },
        {
          "amount": 1,
          "receiver": "Abcde",
          "sender": "174eb42e7bf04a95b26ab32f07c27731"
        }
      ]
    }
  ],
  "length": 2
}
```

For other nodes 5002 and 5001



```
1 {
2   "chain": [
3     {
4       "index": 1,
5       "previous_hash": "0",
6       "proof": 1,
7       "timestamp": "2026-09-10 09:18:13.138437",
8       "transactions": []
9     }
10  ],
11  "length": 1
12 }
13 }
```

Sync nodes



```
1 {
2   "message": "The nodes had different chains so the chain was replaced by the longest one.",
3   "new_chain": [
4     {
5       "index": 1,
6       "previous_hash": "0",
7       "proof": 1,
8       "timestamp": "2026-09-10 09:18:13.265339",
9       "transactions": []
10    },
11    {
12      "index": 2,
13      "previous_hash": "5ca9064bf967aa0e45d31bd93bc958eca8b1b23cdc3f3199c9f35d96f8fd3aca6",
14      "proof": 533,
15      "timestamp": "2026-09-10 09:27:00.134228",
16      "transactions": [
17        {
18          "amount": 10,
19          "receiver": "Shravanya Andhale",
20          "sender": "Ved V Joshi"
21        },
22        {
23          "amount": 5,
24          "receiver": "Nikhil Kadam",
25          "sender": "Shravanya Andhale"
26        }
27      ]
28    }
29  ]
30 }
```

Conclusion:

We developed a cryptocurrency using Python and implemented mining in a simulated three-node blockchain network. Each node maintained its own ledger and synchronized with peers using the Longest Chain Rule to ensure consistency. Mining was performed through Proof-of-Work, securely adding transactions and rewarding miners. This demonstrated key blockchain concepts, including decentralized consensus, transaction validation, and network synchronization.